Boardly – Selenium WebDriver Test Cases

Automated end-to-end tests for **Boardly**, a Kanban-style project board, written in **Python** with **Selenium WebDriver** and run with **pytest**.

Test environment

Item	Value
Language	Python 3
Libraries	`selenium` (WebDriver), `pytest`
Browser	Google Chrome (chromedriver is installed automatically by Selenium Manager)
Application under test	Boardly frontend at `http://localhost:3000`, backend API at `http://localhost:4000`
Email	Mailpit (`http://127.0.0.1:8025`) catches verification emails so the tests can open the links

How to execute the tests

1. Start Docker Desktop.
2. From the project root, start the application:

   ```
   $env:AUTH_RATE_LIMIT = "1000"; docker compose --profile mail up -d --build
   ```

3. Install the test dependencies:

   ```
   cd tests\e2e
   pip install -r requirements.txt
   ```

4. Run the tests in a visible, slowed-down browser:

   ```
   $env:HEADLESS = "0"; $env:SLOW_MO = "1"; python -m pytest -v
   ```

5. To run a single test case, pass its script, for example:

   ```
   python -m pytest -v test_03_invite_teammate.py
   ```

When run together, the test cases run in order (TC-01 → TC-08) and share one browser session. TC-01 to TC-04 can also run on their own: any accounts or group they need that an earlier test didn't create are set up through

the backend API first, so the browser only shows the flow being tested. TC-05 to TC-08 work on the tasks created in TC-04, so they only run after the test cases listed in the summary below have passed in the same run. Every run creates new accounts with random emails, so the suite can be re-run at any time.

Summary

ID	Test case	Script	Runs after
TC-01	Sign up and sign in	`test_01_auth.py`	Runs on its own
TC-02	Create a group (project)	`test_02_create_group.py`	Runs on its own
TC-03	Invite a teammate	`test_03_invite_teammate.py`	Runs on its own
TC-04	Create a task	`test_04_create_task.py`	Runs on its own
TC-05	Assign a task	`test_05_assign_task.py`	TC-03, TC-04
TC-06	Move a task across Kanban columns	`test_06_move_task.py`	TC-04
TC-07	Verify dashboard counts	`test_07_dashboard.py`	TC-05, TC-06
TC-08	Verify calendar deadlines and overdue tasks	`test_08_calendar.py`	TC-04

TC-01: Sign up and sign in

Script: `tests/e2e/test_01_auth.py`

Description

Checks that a new user can create an account, verify their email address, sign out and sign back in, and that a wrong password is rejected. It also creates and verifies a second (teammate) account that later test cases use.

Steps

1. Open `/signup` and enter a full name, a unique email and a password (8+ characters).
2. Click **Create account**.
3. Open the verification link from the email (read from Mailpit) and click **Verify email**.
4. Sign out, open `/login`, enter the same email and password and click **Sign in**.
5. Sign out, open `/login`, enter the same email with a wrong password and click **Sign in**.
6. Repeat steps 1–3 for a second (teammate) account.

Expected output

- After sign-up the user is redirected to `/dashboard` and the **Your groups** page is shown.

- Verification shows "*Your email is verified. You can now send and accept invitations.*"
- Signing in with the correct password redirects to `/dashboard` and shows the user's name.
- Signing in with a wrong password shows a red error message and the browser stays on `/login`.

Python script (Selenium WebDriver)

```python
"""Sign up and sign in."""
from selenium.webdriver.common.by import By

from conftest import (
    BASE_URL, button, sign_in, sign_out, sign_up, verify_email, visible,
)


def test_sign_up_and_sign_in(driver, new_owner, new_teammate):
    sign_up(driver, new_owner)
    verify_email(driver, new_owner)

    sign_out(driver)
    sign_in(driver, new_owner)
    assert new_owner.name in driver.page_source

    # Wrong password is rejected.
    sign_out(driver)
    driver.get(f"{BASE_URL}/login")
    visible(driver, By.ID, "email").send_keys(new_owner.email)
    driver.find_element(By.ID, "password").send_keys("not-the-password")
    button(driver, "Sign in").click()
    visible(driver, By.XPATH, "//div[contains(@class,'bg-red-50')]")
    assert "/login" in driver.current_url

    # The teammate also needs a verified account to accept invitations later.
    sign_up(driver, new_teammate)
    verify_email(driver, new_teammate)
```

TC-02: Create a group (project)

Script: `tests/e2e/test_02_create_group.py`

Description

Checks that a signed-in user can create a new group and becomes its owner, and that the group's Kanban board starts with three empty columns.

Steps

1. Sign in as the owner.
2. On **Your groups**, click **New group**.
3. Enter the group name *ENSF 519 Selenium Team* and click **Create**.
4. Click the group's **Board** link.

Expected output

- A card for *ENSF 519 Selenium Team* appears on the groups page with an **Owner** badge.
- The board shows the columns **To Do**, **In Progress** and **Done**, each with a count of 0.

Python script (Selenium WebDriver)

```
"""Create a group."""
from selenium.webdriver.common.by import By

from conftest import (
    GROUP_NAME, button, column_count, open_board, sign_in, visible,
)


def test_create_group(driver, owner, state):
    sign_in(driver, owner)
    button(driver, "New group").click()
    visible(driver, By.ID, "group-name").send_keys(GROUP_NAME)
    button(driver, "Create").click()

    card = visible(driver, By.XPATH, f"//h3[normalize-
space()='{GROUP_NAME}']/ancestor::div[contains(@class,'rounded-xl')][1]")
    assert "Owner" in card.text
    board_href = card.find_element(By.LINK_TEXT, "Board").get_attribute("href")
    state["group_id"] = board_href.rstrip("/").split("/")[-1]

    open_board(driver, state['group_id'])
    for name in ("To Do", "In Progress", "Done"):
        assert column_count(driver, name) == 0
```

TC-03: Invite a teammate

Script: `tests/e2e/test_03_invite_teammate.py`

Description

Checks that a group owner can invite another user by email, that the invited user sees the invite and can accept it, and that they then appear in the group's member list.

Steps

1. Sign in as the owner and open the group's **Members & settings** page.
2. Enter the teammate's email in **Invite by email** and click **Send invitation**.
3. Sign in as the teammate. On **Your groups**, find the pending invite and click **Accept**.
4. Sign in as the owner again and open **Members & settings**.

Expected output

- A confirmation message containing "*Invitation*" appears after sending.
- The teammate sees "*… invited you to ENSF 519 Selenium Team*". After accepting, the group card appears in their groups list.
- The owner's member list shows the teammate's email with the role **Member**.

Python script (Selenium WebDriver)

```
"""Invite a teammate and have them accept."""
from selenium.webdriver.common.by import By

from conftest import (
    BASE_URL, GROUP_NAME, button, sign_in, visible,
)


def test_invite_teammate(driver, owner, teammate, group_id):
    sign_in(driver, owner)
    driver.get(f"{BASE_URL}/groups/{group_id}/members")
    email_input = visible(driver, By.CSS_SELECTOR, "input[type='email']
[placeholder='teammate@example.com']")
    email_input.send_keys(teammate.email)
    button(driver, "Send invitation").click()
    status = visible(driver, By.XPATH, "//*[@role='status']")
    assert "Invitation" in status.text

    # Teammate accepts the pending invite from their groups page.
    sign_in(driver, teammate)
    invite = visible(driver, By.XPATH, f"//div[contains(@class,'bg-amber-50')][contains(.,
'{GROUP_NAME}')]")
    invite.find_element(By.XPATH, ".//button[normalize-space()='Accept']").click()
    visible(driver, By.XPATH, f"//h3[normalize-space()='{GROUP_NAME}']")

    # Owner now sees the teammate in the member list.
    sign_in(driver, owner)
    driver.get(f"{BASE_URL}/groups/{group_id}/members")
    visible(driver, By.XPATH, f"//li[contains(., '{teammate.email}')][contains(., 'Member')]")
```

TC-04: Create a task

Script: `tests/e2e/test_04_create_task.py`

Description

Checks that tasks can be created from the board, including a due date, and that they appear in the right column with their due date shown.

Steps

1. Sign in as the owner and open the group's board.
2. In the **To Do** column click **+ Add task**.
3. Enter the title *Write test plan*, set **Due date** to yesterday's date and click **Create task**.
4. Repeat for *Prepare demo* (due 3 days from today) and *Set up repo* (no due date).

Expected output

- The **New task** dialog opens and closes after **Create task** is clicked.
- All three cards appear in **To Do** and the column count is 3.
- The *Write test plan* card shows "*Due YYYY-MM-DD*" with yesterday's date.

Python script (Selenium WebDriver)

```
"""Create tasks on the board."""
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC

from conftest import (
    DONE_TASK, OVERDUE_TASK, UPCOMING_TASK, card_xpath, column, column_count, open_board, set_react_value,
sign_in, visible, wait,
)


def test_create_task(driver, owner, group_id):
    sign_in(driver, owner)
    open_board(driver, group_id)
    for title, due in (OVERDUE_TASK, UPCOMING_TASK, DONE_TASK):
        column(driver, "To Do").find_element(By.XPATH, ".//button[normalize-space()='+ Add task']").click()
        dialog = visible(driver, By.CSS_SELECTOR, "[role='dialog']")
        assert "New task" in dialog.text
        dialog.find_element(By.XPATH, ".//label[starts-with(normalize-
space(),'Title')]/input").send_keys(title)
        if due:
            due_input = dialog.find_element(By.XPATH, ".//label[starts-with(normalize-space(),'Due
date')]/input")
            set_react_value(driver, due_input, due.isoformat())
```

```
        dialog.find_element(By.XPATH, ".//button[normalize-space()='Create task']").click()
        wait(driver).until(EC.invisibility_of_element_located((By.CSS_SELECTOR, "[role='dialog']")))
        visible(driver, By.XPATH, card_xpath(title, "To Do"))

    assert column_count(driver, "To Do") == 3
    overdue_card = driver.find_element(By.XPATH, card_xpath(OVERDUE_TASK[0]))
    assert f"Due {OVERDUE_TASK[1].isoformat()}" in overdue_card.text
```

TC-05: Assign a task

Script: `tests/e2e/test_05_assign_task.py`

Description

Checks that a task can be assigned to a group member from the task details dialog, and that the assignment is saved.

Steps

1. Open the group's board and click the *Write test plan* card.
2. In the **Task details** dialog, choose the teammate in the **Assignee** dropdown.
3. Click **Save changes**.
4. Reload the page.

Expected output

- The dialog closes and the card shows the teammate's initials avatar (tooltip = teammate's name).
- After reloading, the avatar is still shown, so the assignment was saved to the server.

Python script (Selenium WebDriver)

```
"""Assign a task to a teammate."""
import pytest
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import Select

from conftest import (
    OVERDUE_TASK, card_xpath, clickable, open_board, visible, wait,
)


@pytest.mark.needs("test_03_invite_teammate", "test_04_create_task")
def test_assign_task(driver, teammate, group_id):
    open_board(driver, group_id)
    clickable(driver, By.XPATH, card_xpath(OVERDUE_TASK[0])).click()
```

```
    dialog = visible(driver, By.CSS_SELECTOR, "[role='dialog']")
    assignee = dialog.find_element(By.XPATH, ".//label[starts-with(normalize-space(),'Assignee')]/select")
    Select(assignee).select_by_visible_text(teammate.name)
    dialog.find_element(By.XPATH, ".//button[normalize-space()='Save changes']").click()
    wait(driver).until(EC.invisibility_of_element_located((By.CSS_SELECTOR, "[role='dialog']")))

    avatar = visible(driver, By.XPATH, f"{card_xpath(OVERDUE_TASK[0])}//span[@title='{teammate.name}']")
    assert avatar.is_displayed()

    # Assignment persists across a reload.
    driver.refresh()
    visible(driver, By.XPATH, f"{card_xpath(OVERDUE_TASK[0])}//span[@title='{teammate.name}']")
```

TC-06: Move a task across Kanban columns

Script: `tests/e2e/test_06_move_task.py`

Description

Checks that cards can be dragged between Kanban columns and that the new column is saved. The drag uses the board's keyboard controls (Space to pick up, arrow keys to move, Space to drop), which Selenium can drive reliably.

Steps

1. Open the group's board.
2. Move *Write test plan* from **To Do** to **In Progress** (focus the card, press Space, Right Arrow, Space).
3. Move *Set up repo* from **To Do** to **Done** (Space, Right Arrow twice, Space).
4. Reload the page.

Expected output

- *Write test plan* appears under **In Progress** and *Set up repo* under **Done**.
- After reloading, the cards are still in those columns, and *Prepare demo* is still in **To Do**.
- Column counts are To Do = 1, In Progress = 1, Done = 1.

Python script (Selenium WebDriver)

```
"""Move tasks across Kanban columns."""
import pytest
from selenium.webdriver.common.by import By

from conftest import (
    DONE_TASK, OVERDUE_TASK, UPCOMING_TASK, card_xpath, column_count, drag_with_keyboard, open_board,
visible, wait,
```

```
)


@pytest.mark.needs("test_04_create_task")
def test_move_task_across_columns(driver, group_id):
    open_board(driver, group_id)
    drag_with_keyboard(driver, OVERDUE_TASK[0], steps_right=1)
    visible(driver, By.XPATH, card_xpath(OVERDUE_TASK[0], "In Progress"))
    wait(driver).until(lambda d: column_count(d, "In Progress") == 1)

    drag_with_keyboard(driver, DONE_TASK[0], steps_right=2)
    visible(driver, By.XPATH, card_xpath(DONE_TASK[0], "Done"))
    wait(driver).until(lambda d: column_count(d, "Done") == 1)

    # Column positions are saved server-side.
    driver.refresh()
    visible(driver, By.XPATH, card_xpath(OVERDUE_TASK[0], "In Progress"))
    visible(driver, By.XPATH, card_xpath(DONE_TASK[0], "Done"))
    visible(driver, By.XPATH, card_xpath(UPCOMING_TASK[0], "To Do"))
    assert [column_count(driver, c) for c in ("To Do", "In Progress", "Done")] == [1, 1, 1]
```

TC-07: Verify dashboard counts

Script: `tests/e2e/test_07_dashboard.py`

Description

Checks that the group dashboard's statistics match the tasks on the board: totals by status, the overdue count, the completion percentage, the per-member team progress table and the deadlines list.

Steps

1. Open the group's **Dashboard** page.
2. Read the stat tiles and the completion chart.
3. Find the teammate's row in the **Team progress** table.
4. In the **Deadlines** section, click the **Overdue** tab.

Expected output

- Total tasks = 3, Completed = 1, In progress = 1, Not started = 1, Overdue = 1.
- The completion chart shows **33% complete**.
- Teammate row: Assigned 1, Completed 0, In progress 1, Overdue 1.
- The **Overdue** tab lists exactly one task: *Write test plan*.

Python script (Selenium WebDriver)

```python
"""Dashboard counts."""
import pytest
from selenium.webdriver.common.by import By

from conftest import (
    BASE_URL, OVERDUE_TASK, clickable, stat_tile, visible,
)


@pytest.mark.needs("test_05_assign_task", "test_06_move_task")
def test_dashboard_counts(driver, teammate, group_id):
    driver.get(f"{BASE_URL}/dashboard/{group_id}")
    visible(driver, By.XPATH, "//p[normalize-space()='Project completion']")

    assert stat_tile(driver, "Total tasks") == 3
    assert stat_tile(driver, "Completed") == 1
    assert stat_tile(driver, "In progress") == 1
    assert stat_tile(driver, "Not started") == 1
    assert stat_tile(driver, "Overdue") == 1
    visible(driver, By.XPATH, "//*[name()='svg'][@aria-label='33% complete']")

    # Team progress row for the teammate: 1 assigned, 0 completed, 1 in progress, 1 overdue.
    row = visible(driver, By.XPATH, f"//tr[.//p[contains(., '{teammate.email}')]]")
    assert [td.text for td in row.find_elements(By.XPATH, "./td")[1:5]] == ["1", "0", "1", "1"]

    # Deadlines list: the overdue tab shows only the overdue task.
    clickable(driver, By.XPATH, "//button[@role='tab'][starts-with(normalize-space(),'Overdue')]").click()
    items = driver.find_elements(By.XPATH, "//section[.//h2[normalize-space()='Deadlines']]//li")
    assert len(items) == 1 and OVERDUE_TASK[0] in items[0].text
```

TC-08: Verify calendar deadlines and overdue tasks

Script: `tests/e2e/test_08_calendar.py`

Description

Checks that the calendar shows each task on its due date, marks past-due unfinished tasks as overdue, lists undated tasks separately, and that the **Overdue only** filter works.

Steps

1. Open the group's **Timeline** page in **Calendar** view (`/groups/<id>/timeline?view=calendar`).
2. Go to the month containing yesterday and find *Write test plan*.
3. Go to the month containing the date 3 days from today and find *Prepare demo*.

4. Check the **No date** section for *Set up repo*.
5. Tick **Overdue only**.

Expected output

- *Write test plan* is in yesterday's day cell, and its tooltip status is **Overdue**.
- *Prepare demo* is on its due date with status **To Do**.
- *Set up repo* is listed under **No date**.
- With **Overdue only** ticked the page shows "*1 of 3 tasks match the filters*": only *Write test plan* is shown and *Prepare demo* is hidden.

Python script (Selenium WebDriver)

```
"""Calendar deadlines and overdue tasks."""
import pytest
from selenium.webdriver.common.by import By

from conftest import (
    BASE_URL, DONE_TASK, OVERDUE_TASK, UPCOMING_TASK, button, calendar_chip, clickable, show_month_of,
visible, wait,
)


@pytest.mark.needs("test_04_create_task")
def test_calendar_deadlines_and_overdue(driver, group_id):
    driver.get(f"{BASE_URL}/groups/{group_id}/timeline?view=calendar")
    button(driver, "Today")

    show_month_of(driver, OVERDUE_TASK[1])
    overdue_chip = wait(driver).until(lambda d: calendar_chip(d, OVERDUE_TASK[0]))
    assert "· Overdue ·" in overdue_chip.get_attribute("title")
    # The chip sits in the cell for its due date.
    day_cell = overdue_chip.find_element(By.XPATH, "./ancestor::div[contains(@class,'min-h-[112px]')][1]")
    assert day_cell.find_element(By.XPATH, ".//span[1]").text == str(OVERDUE_TASK[1].day)

    show_month_of(driver, UPCOMING_TASK[1])
    upcoming_chip = wait(driver).until(lambda d: calendar_chip(d, UPCOMING_TASK[0]))
    assert "· To Do ·" in upcoming_chip.get_attribute("title")

    # Undated task is listed in the "No date" section rather than on a day.
    visible(driver, By.XPATH, f"//section[h3[starts-with(normalize-space(),'No date')]]//button[starts-
with(@title,'{DONE_TASK[0]} ·')]")

    # "Overdue only" filter keeps just the overdue task.
    clickable(driver, By.XPATH, "//label[normalize-space()='Overdue only']/input").click()
```

```
    visible(driver, By.XPATH, "//p[contains(., '1 of 3 tasks match the filters')]")
    show_month_of(driver, OVERDUE_TASK[1])
    assert wait(driver).until(lambda d: calendar_chip(d, OVERDUE_TASK[0]))
    assert calendar_chip(driver, UPCOMING_TASK[0]) is None
```

Appendix: shared helpers (`conftest.py`)

All test scripts import these helpers. The file starts the Chrome WebDriver, creates the test accounts, and provides the reusable steps (sign up, sign in, read the verification email, open the board, keyboard drag-and-drop, read dashboard tiles and calendar chips).

```
"""Shared fixtures and helpers for the Boardly Selenium end-to-end suite.

Requires the stack running with the Mailpit profile so verification emails can be read:
    docker compose --profile mail up -d --build
"""
import json
import os
import re
import time
import urllib.error
import urllib.parse
import urllib.request
import uuid
from dataclasses import dataclass
from datetime import date, timedelta
from pathlib import Path

import pytest
from selenium import webdriver
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.events import AbstractEventListener, EventFiringWebDriver
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

BASE_URL = os.environ.get("BOARDLY_URL", "http://localhost:3000").rstrip("/")
# Used to set up accounts and the group directly when a test file runs on its own.
API_URL = os.environ.get("BOARDLY_API_URL", "http://127.0.0.1:4000/api").rstrip("/")
# 127.0.0.1 rather than localhost: docker-compose binds Mailpit to IPv4 only, and on Windows
# "localhost" tries IPv6 first, which is refused and can stall the request.
MAILPIT_URL = os.environ.get("MAILPIT_URL", "http://127.0.0.1:8025").rstrip("/")
HEADLESS = os.environ.get("HEADLESS", "1") != "0"
# Seconds to pause after each click, typed value and page load, so a person can follow along.
SLOW_MO = float(os.environ.get("SLOW_MO", "0"))
TIMEOUT = 15
```

```python
@dataclass
class Account:
    name: str
    email: str
    password: str = "Password123!"
    verified: bool = False


def new_account(prefix: str) -> Account:
    suffix = uuid.uuid4().hex[:8]
    return Account(name=f"{prefix.title()} {suffix}", email=f"{prefix}-{suffix}@example.com")


@pytest.fixture(scope="session")
def driver():
    options = webdriver.ChromeOptions()
    if HEADLESS:
        options.add_argument("--headless=new")
    options.add_argument("--window-size=1600,1000")
    drv = webdriver.Chrome(options=options)
    if SLOW_MO:
        drv = EventFiringWebDriver(drv, _SlowMo())
    yield drv
    if SLOW_MO:
        time.sleep(SLOW_MO * 3)  # leave the final screen up briefly
    drv.quit()


class _SlowMo(AbstractEventListener):
    def after_navigate_to(self, url, driver):
        time.sleep(SLOW_MO)

    def after_click(self, element, driver):
        time.sleep(SLOW_MO)

    def after_change_value_of(self, element, driver):
        time.sleep(SLOW_MO)


@pytest.fixture(scope="session")
def new_owner() -> Account:
    """The group owner before signing up. test_01 signs it up through the browser."""
    return new_account("owner")


@pytest.fixture(scope="session")
def new_teammate() -> Account:
    return new_account("teammate")


@pytest.fixture
```

```python
def owner(new_owner) -> Account:
    """The owner with a verified account, set up through the API if test_01 didn't run."""
    ensure_verified(new_owner)
    return new_owner


@pytest.fixture
def teammate(new_teammate) -> Account:
    ensure_verified(new_teammate)
    return new_teammate


_STATE: dict = {}


@pytest.fixture(scope="session")
def state() -> dict:
    """Values produced by earlier test files (e.g. the group id) and read by later ones."""
    return _STATE


@pytest.fixture
def group_id(state, owner) -> str:
    """The group test_02 created, or a new one made through the API if test_02 didn't run."""
    if "group_id" not in state:
        token = _api("POST", "/auth/login", {"email": owner.email, "password": owner.password})["token"]
        state["group_id"] = _api("POST", "/groups", {"name": GROUP_NAME}, token)["group"]["id"]
    return state["group_id"]


# ---------- test ordering ----------
# Tests that work on tasks made by earlier files declare it with @pytest.mark.needs("test_04_create_task",
...)
# and are skipped unless those files passed earlier in the same run.

_PASSED: set = set()


def pytest_configure(config):
    config.addinivalue_line("markers", "needs(*files): skip unless these earlier test files passed in this
run")


def pytest_runtest_logreport(report):
    if report.when == "call" and report.passed:
        _PASSED.add(Path(report.nodeid.split("::")[0]).stem)


def pytest_runtest_setup(item):
    marker = item.get_closest_marker("needs")
    missing = [name for name in marker.args if name not in _PASSED] if marker else []
    if missing:
```

```
        pytest.skip(f"Needs {', '.join(missing)} to pass first in the same run")


# ---------- generic helpers ----------

def wait(driver, timeout=TIMEOUT):
    return WebDriverWait(driver, timeout)


def visible(driver, by, value, timeout=TIMEOUT):
    return wait(driver, timeout).until(EC.visibility_of_element_located((by, value)))


def clickable(driver, by, value, timeout=TIMEOUT):
    return wait(driver, timeout).until(EC.element_to_be_clickable((by, value)))


def button(driver, text, timeout=TIMEOUT):
    return clickable(driver, By.XPATH, f"//button[normalize-space()='{text}']", timeout)


def set_react_value(driver, element, value):
    """Set an input's value so React's onChange fires (send_keys on date inputs is locale-dependent)."""
    driver.execute_script(
        """
        const [el, value] = arguments;
        const proto = el.tagName === 'SELECT' ? HTMLSelectElement.prototype
                    : el.tagName === 'TEXTAREA' ? HTMLTextAreaElement.prototype :
HTMLInputElement.prototype;
        Object.getOwnPropertyDescriptor(proto, 'value').set.call(el, value);
        el.dispatchEvent(new Event(el.tagName === 'SELECT' ? 'change' : 'input', { bubbles: true }));
        """,
        element,
        value,
    )


# ---------- auth helpers ----------

def wait_for_dashboard(driver):
    """Wait for the post-auth redirect, failing with the form's error message if one appears."""
    error = (By.CSS_SELECTOR, "form div.bg-red-50")
    wait(driver).until(EC.any_of(EC.url_contains("/dashboard"), EC.visibility_of_element_located(error)))
    if "/dashboard" not in driver.current_url:
        message = driver.find_element(*error).text
        hint = " (auth rate limit hit: restart the backend or set AUTH_RATE_LIMIT)" if "Too many" in message
else ""
        pytest.fail(f"Auth failed: {message}{hint}")
    visible(driver, By.XPATH, "//h1[normalize-space()='Your groups']")


def sign_out(driver):
```

```
    driver.get(f"{BASE_URL}/login")
    driver.execute_script("window.localStorage.clear();")


def sign_up(driver, account: Account):
    sign_out(driver)
    driver.get(f"{BASE_URL}/signup")
    visible(driver, By.ID, "name").send_keys(account.name)
    driver.find_element(By.ID, "email").send_keys(account.email)
    driver.find_element(By.ID, "password").send_keys(account.password)
    button(driver, "Create account").click()
    wait_for_dashboard(driver)


def sign_in(driver, account: Account):
    sign_out(driver)
    driver.get(f"{BASE_URL}/login")
    visible(driver, By.ID, "email").send_keys(account.email)
    driver.find_element(By.ID, "password").send_keys(account.password)
    button(driver, "Sign in").click()
    wait_for_dashboard(driver)


def _mailpit(path: str):
    with urllib.request.urlopen(f"{MAILPIT_URL}{path}", timeout=5) as res:
        return json.loads(res.read())


def verification_link(email: str, timeout=TIMEOUT) -> str:
    """Poll Mailpit for the newest verification email sent to `email` and return its link."""
    query = urllib.parse.quote(f'to:"{email}" subject:"Verify"')
    try:
        _mailpit("/api/v1/info")
    except OSError as err:
        pytest.skip(f"Mailpit not reachable at {MAILPIT_URL} ({err}); run `docker compose --profile mail up
-d`")
    deadline = time.time() + timeout
    while time.time() < deadline:
        try:
            messages = _mailpit(f"/api/v1/search?query={query}").get("messages", [])
            if messages:
                text = _mailpit(f"/api/v1/message/{messages[0]['ID']}")["Text"]
                match = re.search(r"https?://\S+/verify-email#token=\S+", text)
                if match:
                    return match.group(0)
        except OSError:
            pass  # transient connection error; retry until the deadline
        time.sleep(0.5)
    raise AssertionError(f"No verification email for {email} arrived in Mailpit")


def verify_email(driver, account: Account):
```

```
    driver.get(verification_link(account.email))
    button(driver, "Verify email").click()
    visible(driver, By.XPATH, "//*[@role='status'][contains(., 'Your email is verified')]")
    account.verified = True


def _api(method: str, path: str, body: dict, token: str | None = None):
    headers = {"Content-Type": "application/json"}
    if token:
        headers["Authorization"] = f"Bearer {token}"
    req = urllib.request.Request(f"{API_URL}{path}", data=json.dumps(body).encode(), headers=headers,
method=method)
    with urllib.request.urlopen(req, timeout=10) as res:
        return json.loads(res.read())


def ensure_verified(account: Account):
    """Sign up and verify `account` through the API, unless test_01 already did it in the browser."""
    if account.verified:
        return
    credentials = {"email": account.email, "password": account.password}
    try:
        user = _api("POST", "/auth/signup", {"name": account.name, **credentials})["user"]
    except urllib.error.HTTPError as err:
        if err.code != 409:  # 409: already signed up, e.g. by a test_01 that failed before verifying
            raise
        user = _api("POST", "/auth/login", credentials)["user"]
    if not user["emailVerified"]:
        token = verification_link(account.email).split("#token=", 1)[1]
        _api("POST", "/auth/verify-email", {"token": token})
    account.verified = True


# ---------- shared test data ----------

GROUP_NAME = "ENSF 519 Selenium Team"
TODAY = date.today()
OVERDUE_TASK = ("Write test plan", TODAY - timedelta(days=1))
UPCOMING_TASK = ("Prepare demo", TODAY + timedelta(days=3))
DONE_TASK = ("Set up repo", None)


# ---------- page helpers ----------

def column(driver, name):
    return visible(driver, By.XPATH, f"//section[h2[starts-with(normalize-space(), '{name}')]]")


def column_count(driver, name):
    return int(column(driver, name).find_element(By.XPATH, "./h2/span").text)
```

```python
def card_xpath(title, column_name=None):
    scope = f"//section[h2[starts-with(normalize-space(), '{column_name}')]]" if column_name else ""
    return f"{scope}//div[@data-rfd-draggable-id][.//p[normalize-space()='{title}']]"


def open_board(driver, group_id):
    driver.get(f"{BASE_URL}/board/{group_id}")
    column(driver, "To Do")


def stat_tile(driver, label):
    return int(visible(driver, By.XPATH, f"//div[contains(@class,'panel')][p[normalize-
space()='{label}']]/p[2]").text)


def drag_with_keyboard(driver, title, steps_right):
    """@hello-pangea/dnd supports keyboard dragging: Space to lift, arrows to move, Space to drop."""
    card = visible(driver, By.XPATH, card_xpath(title))
    driver.execute_script("arguments[0].focus();", card)
    actions = ActionChains(driver).send_keys(Keys.SPACE).pause(0.4)
    for _ in range(steps_right):
        actions.send_keys(Keys.ARROW_RIGHT).pause(0.4)
    actions.send_keys(Keys.SPACE).perform()


def calendar_chip(driver, title):
    chips = driver.find_elements(By.XPATH, f"//button[starts-with(@title, '{title} ·')]")
    return next((c for c in chips if c.is_displayed()), None)


def show_month_of(driver, day: date):
    button(driver, "Today").click()
    months = (day.year - TODAY.year) * 12 + day.month - TODAY.month
    label = "Next month" if months > 0 else "Previous month"
    for _ in range(abs(months)):
        clickable(driver, By.CSS_SELECTOR, f"button[aria-label='{label}']").click()
```